

AdPulse v2

AI-Powered Ad Campaign Performance Analyzer

Team Explanation Guide

Team 6 -- Purple Cohort

Graeme Ampeire | Soumya Thareja | Tiffany Nakamitsu | Akanksha Singh

MSIS 521 -- IT and Marketing in the New Economy

Winter 2025 | University of Washington

This guide explains the AdPulse project in two ways: a plain-language overview that any team member can reference for the presentation, and a deeper technical breakdown of every module, algorithm, and course concept we implemented.

PART 1: Plain-Language Overview

What Is AdPulse?

AdPulse is an interactive web dashboard that analyzes Super Bowl commercials using artificial intelligence. Think of it as a smart assistant that watches all 247 Super Bowl ads from 2000 to 2020 and tells you which ones worked best, why they worked, and what patterns brands follow.

What Problem Does It Solve?

Super Bowl ads cost over \$5 million for 30 seconds. Brands spend enormous budgets but often rely on gut feeling to decide what makes a good ad. AdPulse replaces guesswork with data. It looks at real YouTube engagement (views, likes, comments) and uses AI to find patterns that humans might miss.

What Does It Actually Do?

- **Ranks every ad** by a composite engagement score combining views, likes, and comments
- **Analyzes sentiment** of ad descriptions using two different AI methods (VADER and BERT) to determine if the tone is positive, negative, or neutral
- **Discovers hidden topics** in ad copy using topic modeling, showing what themes brands talk about
- **Groups similar ads together** using clustering algorithms, revealing which brands have similar ad strategies (for example, beer brands all advertise similarly)
- **Maps brand relationships** as a visual network graph where connected brands share similar approaches
- **Evaluates crowd wisdom** by testing whether the Super Bowl ad ecosystem meets the scientific conditions for collective intelligence
- **Computes marketing efficiency metrics** like cost per view and return on investment, ranking brands the same way the Air France SEM case study ranked search publishers
- **Tracks search trends** by pulling Google Trends data to see how ad buzz translates to real search interest

How Do You Use It?

Run one command in the terminal (`streamlit run app.py`) and a web browser opens with an 11-page interactive dashboard. You can filter by brand, year, or ad features. Every chart is interactive -- you can hover for details, zoom, and pan. No coding required to explore the data once the app is running.

Key Findings (Good for the Presentation)

- **Show your product early.** Ads that display the product quickly get 70% more views than those that don't. Clarity beats cleverness.

- **Celebrities are overrated.** Despite costing more, celebrity ads actually average 54% fewer views. The story and product matter more than the star.
- **Brands copy each other.** Our clustering analysis shows beer brands (Bud Light, Budweiser) and car brands (Toyota, Kia, Hyundai) follow nearly identical ad strategies year after year.
- **The crowd is only somewhat wise.** We scored the Super Bowl ad ecosystem at 41 out of 100 on the Wisdom of Crowds framework. Brands show herding behavior rather than true strategic diversity.
- **Efficiency varies wildly.** The cost per YouTube view ranges from under \$1 to over \$50 across brands, revealing massive differences in how well each brand converts TV exposure into digital engagement.

How Many Course Concepts Does It Cover?

AdPulse implements 21 techniques from 5 of our 6 course sessions. These range from Google Trends (Session 1) and TF-IDF word embeddings (Session 2) to BERT deep learning and K-Means clustering (Session 3), SEM metrics like CPV and Quality Score (Session 5), and network centrality analysis with Wisdom of Crowds (Session 6). The Methodology page in the app has a full table mapping each technique to its course session.

PART 2: Technical Deep Dive

Project Architecture

AdPulse is a Python application built on the Streamlit framework. The codebase is organized into modular Python files, each handling a specific analysis domain. The main entry point (`app.py`, ~1,500 lines) orchestrates an 11-page dashboard that imports from five analysis modules.

File	Lines	Purpose
<code>app.py</code>	~1,530	Streamlit dashboard, 11 pages, all visualizations
<code>analysis.py</code>	~270	VADER sentiment, LDA topics, engagement scoring, Google Trends
<code>clustering.py</code>	~290	TF-IDF, Word2Vec, t-SNE, cosine/Jaccard similarity, K-Means, hierarchical clustering, BERT
<code>network_analysis.py</code>	~365	Brand/ad networks, 4 centrality measures, Pyvis visualization, Wisdom of Crowds
<code>sem_analysis.py</code>	~300	SEM metrics (CPV, CPE, ROI), Quality Score proxy, keyword strategy, publisher comparison
<code>data_utils.py</code>	~115	Data loading, cleaning, text preprocessing with NLTK

Dataset

We use the FiveThirtyEight/TidyTuesday Super Bowl Ads dataset: 247 ads from 10 major brands spanning 2000-2020. Each row contains 25 columns including 7 boolean ad characteristics (funny, celebrity, patriotic, danger, animals, use_sex, show_product_quickly), YouTube engagement metrics (views, likes, dislikes, comments, favorites), and video metadata (title, description, thumbnail URL, publish date). Of the 247 ads, 207 have non-null descriptions suitable for NLP analysis.

Module: Engagement & Feature Impact (v1)

The `EngagementAnalyzer` class normalizes `view_count`, `like_count`, and `comment_count` to 0-1 range, then computes a weighted composite score (40% views, 35% likes, 25% comments). Feature impact analysis compares mean views for ads with vs. without each boolean feature, computing lift percentages. This reveals that `show_product_quickly` delivers +70% views lift while `celebrity` usage shows -54%.

Module: Sentiment Analysis (v1 + v2 BERT)

v1 uses NLTK's VADER (Valence Aware Dictionary for sEntiment Reasoning), a rule-based sentiment analyzer that outputs compound, positive, negative, and neutral scores. v2 adds optional BERT sentiment using the `nlptown/bert-base-multilingual-uncased-sentiment` model, which classifies text into 1-5 star ratings. The `compare_vader_bert()` function aligns both scales and

computes agreement rate. A SentimentEvaluator validates outputs: it checks distribution skew (flags if >95% single class) and tests 5 known-sentiment samples for 80%+ accuracy.

Module: LDA Topic Modeling (v1)

The TopicModeler uses gensim's LDA (Latent Dirichlet Allocation) to discover latent themes in ad descriptions. Text preprocessing includes lowercasing, tokenization, stopword removal, and lemmatization via NLTK. The model extracts 5 topics, each represented by its top 10 weighted terms. Topic distinctness is measured by average pairwise cosine distance between topic vectors (score: 0.312). Interactive word clouds and topic-document distributions are rendered in Plotly.

Module: Text Analysis & Clustering (v2 -- Sessions 2 & 3)

TF-IDF Vectorization

scikit-learn's TfidfVectorizer transforms 207 ad descriptions into a term-frequency inverse-document-frequency matrix (193 docs x 500 features after filtering). Per-brand fingerprints show each brand's most distinctive vocabulary. Top terms are extracted by averaging TF-IDF scores across each brand's ads.

Word2Vec Embeddings

gensim's Word2Vec trains on the tokenized ad corpus (vocab ~800 words, 100-dimensional vectors, window=5, min_count=2). Document embeddings are computed by averaging constituent word vectors. The dashboard includes a word similarity explorer where users enter a term and see semantically related words.

t-SNE Visualization

scikit-learn's t-SNE (t-distributed Stochastic Neighbor Embedding) projects the high-dimensional Word2Vec document embeddings down to 2D for visualization. The resulting scatter plot colors points by brand, revealing spatial clusters of similar ad content.

Cosine & Jaccard Similarity

Cosine similarity measures textual similarity between all ad pairs using TF-IDF vectors (rendered as an interactive heatmap). Jaccard similarity compares brands by their boolean feature overlap: $|\text{intersection}| / |\text{union}|$ of feature usage patterns.

K-Means Clustering

K-Means groups ads into k clusters based on combined feature vectors (TF-IDF + boolean features + normalized engagement). The dashboard lets users select k (2-10) and displays cluster profiles showing the dominant features and average engagement per cluster. Results are visualized on the t-SNE scatter plot.

Hierarchical Clustering

Ward-linkage hierarchical clustering is applied to brand-level average feature profiles. The resulting dendrogram reveals which brands have the most similar ad strategies. For example, Bud Light and Budweiser cluster tightly, as do Toyota, Kia, and Hyundai.

Module: Network & Crowd Intelligence (v2 -- Session 6)

Brand Strategy Network

A NetworkX graph with 10 nodes (brands) and edges weighted by cosine similarity of their average feature profiles + engagement patterns. Edges are only drawn above a configurable similarity threshold (default 0.3). Node size represents total engagement. The graph is rendered as an interactive force-directed visualization using Pyvis, embedded directly in the Streamlit dashboard via an HTML component.

Ad Content Network

A second network connects individual ads (not brands) based on TF-IDF cosine similarity. With 193 nodes and up to 500 edges (capped for performance), this reveals content clusters that cross brand boundaries.

Centrality Analysis

Four centrality measures are computed on the ad network: degree centrality (how connected an ad is), betweenness centrality (how often it bridges different clusters), closeness centrality (how accessible it is to all other ads), and PageRank (Google's algorithm measuring influence based on connection quality). Results are aggregated by brand and displayed as comparative bar charts.

Wisdom of Crowds

We evaluate the Super Bowl ad ecosystem against James Surowiecki's four conditions for collective intelligence, each scored 0-100:

- **Diversity of Opinion** -- Shannon entropy of feature combinations across all ads. Higher entropy = more diverse strategies.
- **Independence** -- Average inter-brand correlation of feature usage over time. Low correlation = brands making independent choices.
- **Decentralization** -- Inverse Gini coefficient of engagement distribution. Low Gini = engagement spread across many brands.
- **Aggregation** -- Rank correlation between crowd-voted engagement (views) and a multi-metric quality proxy (engagement rate + like ratio + comments). High correlation = crowd effectively aggregates quality signals.

The overall score (~41/100) suggests moderate crowd wisdom with room for improvement, particularly in strategic diversity and independence.

Module: SEM Performance Lab (v2 -- Session 5)

This module reframes Super Bowl ads through the lens of search engine marketing metrics, inspired by the Air France SEM case study. Since the dataset lacks actual budget data, we estimate costs using industry benchmarks for Super Bowl ad pricing (\$2.1M in 2000 scaling to \$5.6M in 2020) and viewership (88M to 114M).

- **CPV (Cost Per View)** -- Estimated ad cost divided by YouTube view count. Measures how efficiently a brand converts TV spend into digital views.
- **CPE (Cost Per Engagement)** -- Estimated cost divided by total engagements (likes + comments). Measures cost efficiency of driving meaningful interaction.

- **Quality Score Proxy (0-10)** -- Weighted combination of engagement rate rank (35%), like ratio rank (35%), and view count rank (30%). Analogous to Google Ads Quality Score.
- **Engagement ROI** -- Composite engagement score per million dollars spent. The primary efficiency metric.
- **Digital CTR Equivalent** -- YouTube views divided by estimated TV impressions. Measures how well TV exposure converts to digital interest.

The Publisher-Style Brand Comparison ranks all 10 brands in a table by CPV, CPE, ROI, and Quality Score, mirroring how the Air France case ranked Google, Yahoo, MSN, and other search publishers. Keyword Strategy Analysis uses TF-IDF to extract top terms per brand and categorizes them into Emotional, Action, Product, and Brand buckets, then correlates category usage with engagement outcomes.

Module: Google Trends Integration (v1 -- Session 1)

The TrendAnalyzer uses the pytrends library to pull Google Trends data for selected brands. This connects ad performance to real-world search behavior, showing whether Super Bowl ad buzz translates into sustained search interest. Interactive time series charts display relative search volume over configurable date ranges.

Course Concept Mapping

The table below maps every technique implemented in AdPulse to its corresponding MSIS 521 course session and the module where it appears in the app.

Technique	Course Session	App Module
Google Trends API	S1: Web Analytics	Search Trends page
TF-IDF Vectorization	S2: Word Embedding	Text Analysis & Clustering
Word2Vec Embeddings	S2: Word Embedding	Text Analysis & Clustering
t-SNE Visualization	S2: Word Embedding	Text Analysis & Clustering
VADER Sentiment	S3: Text Analysis	Sentiment Analysis page
BERT Sentiment	S3: Text Analysis / LLMs	Text Analysis & Clustering
LDA Topic Modeling	S3: Text Analysis	Topic Discovery page
Cosine Similarity	S3: Text Analysis	Text Analysis & Clustering
Jaccard Similarity	S3: Text Analysis	Text Analysis & Clustering
K-Means Clustering	S3: Text Analysis	Text Analysis & Clustering
Hierarchical Clustering	S3: Text Analysis	Text Analysis & Clustering
Dendrogram	S3: Text Analysis	Text Analysis & Clustering
CPV / CPE Metrics	S5: SEM/SEO	SEM Performance Lab
Quality Score Proxy	S5: SEM/SEO	SEM Performance Lab
ROI / ROA Analysis	S5: SEM/SEO	SEM Performance Lab
Keyword Strategy	S5: SEM/SEO	SEM Performance Lab
Publisher Comparison	S5: SEM/SEO	SEM Performance Lab
NetworkX Graph Analysis	S6: Social Networks	Network & Crowd Intel
Degree Centrality	S6: Social Networks	Network & Crowd Intel
Betweenness Centrality	S6: Social Networks	Network & Crowd Intel
Closeness / PageRank	S6: Social Networks	Network & Crowd Intel
Wisdom of Crowds	S6: Social Networks	Network & Crowd Intel

How to Run the App

Prerequisites: Python 3.9+ with pip. All dependencies are listed in `requirements.txt`.

1. Open a terminal and navigate to the AdPulse project directory

2. Install dependencies: `pip install -r requirements.txt`
3. Run the app: `streamlit run app.py`
4. The dashboard opens automatically in your default browser at `localhost:8501`
5. Use the sidebar to navigate between all 11 pages
6. Optional: Install transformers and torch for BERT sentiment (large download, ~1GB)

Presentation Tips

- Start with the problem slide to set context -- \$5M+ per ad, brands need data not guesswork
- When showing the dashboard live, start on the Dashboard page for the overview, then jump to Feature Impact (the 70% lift stat is a crowd-pleaser)
- The dendrogram and network graph are the most visually impressive -- great for the technical demo portion
- Wisdom of Crowds is unique and shows deep course engagement; explain Surowiecki's 4 conditions briefly
- The SEM Performance Lab directly connects to the Air France case study from Session 5 -- mention this link explicitly
- End with the Methodology page showing 21 techniques across 5 sessions to reinforce breadth